

FOR SOFTWARE ENGINEERS: THE DEEP DIVE

Distributed Transactions

What @Transactional actually covers

Rolled back, and the customer is still charged. So what, exactly, was transactional?

A method writes to the database, charges a card over HTTP, and writes again. There is an `@Transactional` on top of it. The second write fails, Spring does exactly what it promised and rolls the database back, and the card stays charged.

A transaction boundary in your code is not the same thing as an atomic business operation. Atomicity only extends across resources that actually participate in the transaction protocol. Everything on this page is a consequence of that sentence.

Every quote here is verbatim from the vendor reference, fetched September 2026, and every number is a measurement taken on one laptop rather than a figure repeated from somewhere else.

In scope

- What participates in a transaction, and what only looks like it does
- The measured cost of a remote call inside the boundary
- Two phase commit, including the in-doubt window
- Why Kafka is not an XA resource, and what follows from that
- The outbox, and sagas whose compensation also fails

Out of scope

- A tour of Spring transaction attributes
- Any claim that microservices caused this. Two resources cause this
- Exactly once delivery, which you are not getting
- Choosing a pattern before naming the invariant it protects

If you read one row of the table below, read the 200 request one. That is the moment a database concern becomes an availability concern.

01 A remote call inside the boundary, measured

40 requests, call INSIDE the boundary	1049ms, 40 served, 0 refused
40 requests, call OUTSIDE	212ms, 40 served, 0 refused
200 requests, call INSIDE the boundary	1232ms, 47 served, 153 REFUSED
200 requests, call OUTSIDE	234ms, 200 served, 0 refused
What changed between the pairs	nothing but where the boundary was drawn

Ten connections, a provider that takes 200ms, a 1s connection timeout, and the identical workload run twice. The only variable is whether the 200ms happens while a pooled connection is held. Timing measurements on a laptop move a few percent between runs; the shape does not.

02 What actually participated

THE TWO DATABASE STATEMENTS: yes. One connection, one database transaction, and the rollback undid both.

THE HTTP CALL: no. It is not a resource manager, it never enlisted in anything, and your rollback is a local event it has never heard of. A Kafka publish in the same method: also no.

The method boundary is a boundary in YOUR process. It means almost nothing to the other system, and the most expensive misunderstanding in this whole subject is assuming otherwise because the lines sit next to each other on screen.

AND THE PROXY DETAIL, quoted because it is precise: "In proxy mode (which is the default), only external method calls coming in through the proxy are intercepted." Self invocation does not start a transaction. A real trap, and the small thing on this page.

In proxy mode (which is the default), only external method calls coming in through the proxy are intercepted.

`Spring Framework reference, declarative transaction management`

03 A remote call inside the boundary is an availability problem

A connection pool is a fixed resource, and a transaction holds one connection for its entire duration. Put a remote call inside the boundary and the duration of your database transaction becomes the duration of a network you do not control.

AT 40 REQUESTS nothing is refused and the work takes five times as long. No errors, no logs, a slower endpoint that reads as normal variance. That is the version that gets dismissed, and it means the system is already running with no headroom.

AT 200 REQUESTS the wait for a connection exceeds the timeout and 153 of them are REFUSED. Same pool, same database, same queries.

When the provider gets slow you do not get slow. You start refusing, and you refuse everybody, including the requests that were never going near that provider.

200 concurrent requests, ten connections, 200ms provider: 47 served and 153 refused with the call inside the boundary, 200 served and 0 refused with it outside.

`Run on the machine that built this sheet, not quoted`



04 The warning in the framework own documentation

Quoted in full, because it is the vendor describing the failure and most people using this propagation level have never read it:

"The resources attached to the outer transaction will remain bound there while the inner transaction acquires its own resources such as a new database connection. This may lead to exhaustion of the connection pool and potentially to a deadlock if several threads have an active outer transaction and wait to acquire a new connection for their inner transaction, with the pool not being able to hand out any such inner connection anymore."

"Do not use PROPAGATION_REQUIRES_NEW unless your connection pool is appropriately sized, exceeding the number of concurrent threads by at least 1."

Do not use PROPAGATION_REQUIRES_NEW unless your connection pool is appropriately sized, exceeding the number of concurrent threads by at least 1.

Spring Framework reference, transaction propagation

05 Two phase commit, including the part that gets left out

A coordinator and resource managers that can promise before they act. PREPARE: each resource writes enough durable state to be able to commit later, and answers yes or no. COMMIT: if everyone said yes, the coordinator tells them all to go.

It genuinely gives atomicity across two databases. That is real engineering, not a punchline.

THE WINDOW. Between prepare and commit, every resource has said yes, is holding locks, and is waiting. If the coordinator dies there, those transactions are IN DOUBT: not committed, not rolled back, locks still held, and a human involved.

That is not a reason never to use it. It is the reason the coordinator itself has to be highly available, which is a system you now operate.

In doubt means not committed and not rolled back, with locks still held.

Operational practice, not vendor documentation

06 Kafka is not an XA resource, and that changes the advice

THE CLAIM YOU WILL HEAR: you could use XA across your database and Kafka, but it is slow, so do not. The first half is not true, and the correction matters more than the advice.

KAFKA TRANSACTIONS ARE REAL. A transactional id, a transaction coordinator, atomicity across writes to multiple partitions and topics, and the consumer offsets committed with them. That is what exactly once processing is built on.

But it is the Kafka protocol. The producer is not an XAResource, it does not enlist in your JTA transaction manager, and it does not take part in a two phase commit with your database. The KafkaProducer javadoc mentions no XA and no external resource managers.

SO THE HONEST SENTENCE IS: the reason you cannot make a database write and a Kafka publish atomic is not that it would be a bad idea. The protocol to do it does not exist there.

The transactional producer allows an application to send messages to multiple partitions (and topics!) atomically.

KafkaProducer javadoc

07 Two transaction managers are not two phase commit

Spring can SYNCHRONISE a Kafka transaction with a database one. The reference states the order plainly: "When the method exits, the database transaction will commit followed by the Kafka transaction." Two managers, two commits, two decisions.

And then the sentence that settles the argument, quoted in full:

"if the commit fails on the synchronized transaction (after the primary transaction has committed), the exception will be thrown to the caller. Previously, this was silently ignored (logged at debug level). Applications should take remedial action, if necessary, to compensate for the committed primary transaction."

READ WHAT THAT ADMITS. The database committed. The Kafka commit failed. The advice is that your application compensates. That is not atomicity, it is a documented partial failure with a documented owner, and the owner is you.

At the time of writing, September 2026, ChainedKafkaTransactionManager has been deprecated since version 2.7.

Applications should take remedial action, if necessary, to compensate for the committed primary transaction.

Spring for Apache Kafka reference, Transactions



08 What replaces it, and the saga nobody draws

THE OUTBOX stops needing two resources. Write the business row and a row recording the intent to publish, in ONE transaction, in the database you already had. Then a separate process reads that table and publishes, retrying until it succeeds. That is at least once delivery plus idempotent consumers: trading an impossible guarantee for a possible one, deliberately.

SAGAS, honestly. The cartoon is reserve inventory, take payment, payment fails, release inventory, everybody goes home. The real one is: reserve SUCCESS, pay SUCCESS, ship FAILED, refund the payment ALSO FAILED. What now?

There is no rollback. There was never a rollback. A compensating action is another distributed operation with every failure mode of the original: it can time out, it can execute and lose the response, and it can genuinely fail.

A SAGA DOES NOT RESTORE ATOMICITY. It is a protocol for driving the system toward an acceptable business state after a partial success, and saying so plainly is the difference between a design and a diagram.

A compensating action needs idempotency, durable retry state, observable status, a bounded retry policy, reconciliation, and a terminal state meaning a human must look at it.

`Operational practice, not vendor documentation`

09 Choose the invariant, not the pattern

The senior answer here is not a pattern. It is a question: which invariant are you protecting?

A payment is captured at most once. A refund never exceeds the capture. Ledger entries balance. Name the two or three that must never be violated.

Then decide, deliberately, what you are willing to be eventually consistent about. Because you will be eventually consistent about something. The only choice is whether you chose it, or whether it chose you at three in the morning.

Naming the invariant before the pattern is what separates a design from a diagram.

`Operational practice, not vendor documentation`

Before you call an operation atomic

1. Which of these effects are on resources that join the transaction protocol?
2. Is there a remote call inside the boundary, and what is its p99?
3. How big is the connection pool, and how many concurrent requests hold one?
4. Does anything here use REQUIRES_NEW, and is the pool sized for it?
5. If two resources must both change, which one commits first, and what happens if the second fails?
6. Who compensates when the primary has committed and the secondary has not?
7. Is the compensating action idempotent, retried, and observable?
8. What is the terminal state when compensation keeps failing?
9. Which invariant is this protecting, and which am I choosing to relax?
10. And if the coordinator died right now, what would be holding locks?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of the four your team actually does.

Study findings are attributed to their authors and were read at the primary source rather than from a summary. Product and company names appear only to identify what was studied. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-15 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.